

**SIMATS ENGINEERING**  
**ASSESSMENT TOOL 1: Constraint-Based Problem Solving**

**COURSE NAME:** Cloud Computing and Big Data Analytics      **COURSE CODE:** CSA1522

|                 |                    |
|-----------------|--------------------|
| Register Number | 192572101          |
| Name            | M.Kyathi Sai Priya |

**COURSE OUTCOME COVERED**

**CO5:** *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*  
(BL5)

Dave's Taxonomy: Articulation (Level 4)  
Question Count: 5

**Total Marks: 50**

**Code of Conduct**

I M.Kyathi Sai Priya(192572101)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student M.Kyathi Sai Priya

**Assessment Tasks**

**Constraint-Based Problem Solving**

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

**Constraint-Based Problem Solving Rubric**

| Criteria                     | Weightage | Level 4 - Exemplary                          | Level 3 - Proficient        | Level 2 - Developing        | Level 1 - Beginning           |
|------------------------------|-----------|----------------------------------------------|-----------------------------|-----------------------------|-------------------------------|
| Understanding of Constraints | 25%       | All technical constraints are identified and | Most constraints identified | Some constraints identified | Constraints poorly understood |

|                                |     |                                                                  |                      |                         |                          |
|--------------------------------|-----|------------------------------------------------------------------|----------------------|-------------------------|--------------------------|
|                                |     | prioritized accurately                                           |                      |                         |                          |
| System Design Quality          | 25% | Design is robust, feasible, and aligned to constraints           | Reasonable design    | Basic design with gaps  | Weak design              |
| Application of Hadoop Concepts | 20% | Uses HDFS, MapReduce, YARN, and integration concepts effectively | Mostly effective use | Partial use of concepts | Weak concept application |
| Justification                  | 15% | Strong technical reasoning for all choices                       | Good justification   | Basic justification     | Little justification     |
| Clarity                        | 15% | Clear, well-structured, and professional answer                  | Mostly clear         | Somewhat unclear        | Poorly structured        |

## 1. Question

**Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.**

## Answer

For a media platform storing videos, images, audio, and metadata, **HDFS** can be designed with a replication factor of **3** to provide fault tolerance while controlling storage overhead.

## Proposed Design

### Architecture:

Media Uploads → Ingestion Layer → HDFS → Processing → Metadata/Analytics → Applications

### 1. HDFS Cluster

The cluster should contain:

- **1 Active NameNode**
- **1 Standby NameNode**
- Multiple DataNodes
- JournalNodes for NameNode metadata

- ZooKeeper for automatic failover

The Standby NameNode improves availability if the Active NameNode fails.

## 2. Replication Factor

Set the HDFS replication factor to **3**.

For example:

Block → DataNode 1 + DataNode 2 + DataNode 3

This protects data against multiple node/disk failures without exceeding the given limit.

## 3. Cost Efficiency

To reduce storage cost:

- Use **3× replication only for critical data**.
- Compress metadata and text files.
- Store large media files using suitable block sizes.
- Use erasure coding for suitable large, less frequently accessed datasets where supported.
- Move old media to cheaper storage tiers when possible.
- Delete unnecessary temporary files.

## 4. Block Placement

HDFS should place replicas across different racks.

Example:

Replica 1 → Rack 1

Replica 2 → Rack 2

Replica 3 → Rack 2/3

Rack awareness prevents a complete rack failure from destroying all copies.

## 5. Availability

Availability above **99.9%** can be supported through:

- Active/Standby NameNodes

- Automatic failover
- Replication factor 3
- Rack-aware placement
- Regular health monitoring
- DataNode heartbeat monitoring

## 6. Data Organization

Data can be organized as:

/media/

  /video/

  /audio/

  /images/

  /metadata/

  /archive/

Large media files are divided into HDFS blocks and distributed across DataNodes.

### Justification

This design provides high availability without exceeding the replication limit. Replication, rack awareness, and NameNode failover protect against failures, while compression, archival, and selective use of replication/erasure coding control storage growth.

## 2. Question

**A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.**

### Answer

The objective is to process large log files within a fixed **20-minute batch window** while using a limited number of cluster nodes. The job should therefore use parallel Map and Reduce tasks efficiently.

### Proposed Workflow

## **Architecture:**

Log Files → HDFS → Input Splitting → Mapper → Shuffle & Sort → Reducer → Output

### **1. Input Storage**

Store log files in HDFS.

Example:

/logs/2026/08/24/

Files should be distributed across DataNodes.

### **2. Input Splits**

The log data is divided into input splits.

For example:

Large Log File → Split 1 + Split 2 + Split 3 + ...

Each split is processed independently by a Mapper.

### **3. Mapper**

The Mapper reads each log record and generates useful key-value pairs.

Example:

Input:

UserA /login

UserA /purchase

Mapper Output:

UserA → 1

UserA → 1

### **4. Shuffle and Sort**

MapReduce automatically groups identical keys.

UserA → [1,1,1,1]

UserB  $\rightarrow$  [1,1,1]

## 5. Reducer

Reducers aggregate the values.

UserA  $\rightarrow$  4

UserB  $\rightarrow$  3

## 6. Task Distribution

With limited nodes:

- Use one task per available CPU core where appropriate.
- Run multiple Mapper tasks in parallel.
- Use a smaller number of Reducers based on available resources.
- Keep processing close to the data to reduce network traffic.

Example:

Node 1  $\rightarrow$  Mapper 1 + Mapper 2

Node 2  $\rightarrow$  Mapper 3 + Mapper 4

Node 3  $\rightarrow$  Mapper 5 + Mapper 6

Node 4  $\rightarrow$  Reducer 1 + Reducer 2

## 7. Optimization for 20-Minute Window

To meet the deadline:

- Use appropriate HDFS block sizes.
- Avoid unnecessary data movement.
- Use a **Combiner** to reduce intermediate data.
- Compress intermediate map output.
- Avoid excessive reducers.
- Use speculative execution for slow tasks.
- Monitor task completion time.

## 8. Combiner

If Mapper output is:

UserA → 1

UserA → 1

UserA → 1

The Combiner can produce:

UserA → 3

This reduces network traffic during shuffle.

## 9. Time Management

The job should be designed with a target below 20 minutes, for example:

- Input/Map: 7 minutes
- Shuffle/Sort: 5 minutes
- Reduce: 5 minutes
- Finalization: 2 minutes

This leaves a small safety margin.

## Conclusion

A parallel MapReduce job using **HDFS data locality, optimized Mapper/Reducer counts, combiners, compression, and speculative execution** can process log data efficiently within the 20-minute batch window despite limited cluster nodes.

## 3. Question

**A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.**

## Answer

A YARN cluster must allocate resources among ETL, reporting, and machine-learning workloads while giving ETL higher priority and maintaining fairness.

## Proposed Scheduler

A **Capacity Scheduler** or **Fair Scheduler-style queue policy** can be used.

## Queue Structure

Root

|— ETL → High Priority

|— Reporting → Medium Priority

|— ML → Normal Priority

### 1. ETL Queue

ETL jobs should receive higher priority because they prepare data for downstream analytics.

Example:

**ETL → 50% minimum capacity**

When ETL jobs are waiting, they receive resources preferentially.

### 2. Reporting Queue

Reporting jobs require predictable response times.

Example:

**Reporting → 20–30% capacity**

This prevents reporting workloads from being completely blocked by ML jobs.

### 3. Machine Learning Queue

ML jobs are generally long-running and resource intensive.

Example:

**ML → 20–30% capacity**

ML jobs can use additional unused resources when ETL and reporting workloads are low.

### 4. Fairness

Although ETL has higher priority, the scheduler should prevent starvation.

For example:



ETL → High Priority

Reporting → Medium Priority

ML → Lower Priority

Unused resources can be dynamically shared among queues.

## **5. Node Failure Recovery**

YARN detects node failures using NodeManager heartbeats.

If a NodeManager stops responding:

Node Failure → ResourceManager Detection → Task Reallocation → Another Node

Failed containers are restarted on healthy nodes.

## **6. Resource Limits**

Each queue should have:

- Minimum capacity
- Maximum capacity
- User limits
- Application limits

This prevents a single workload from consuming the entire cluster.

## **7. Preemption**

If ETL requires resources urgently, lower-priority workloads can be preempted within configured limits.

For example:

ML running → ETL arrives → Some ML resources released → ETL executes

## **8. Fault Tolerance**

The design should include:

- ResourceManager High Availability
- NodeManager health monitoring

- Container retry
- ApplicationMaster recovery
- Checkpointing for long-running ML jobs

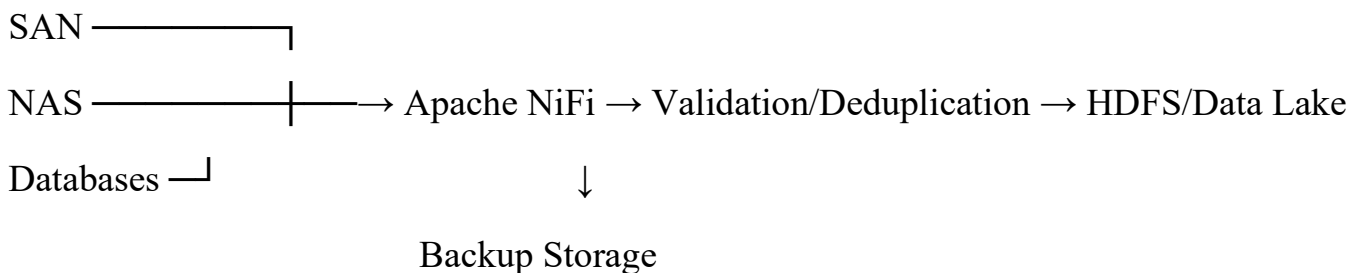
#### 4. Question

**An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.**

#### Answer

The enterprise requires a controlled data-ingestion architecture that supports different storage sources while minimizing duplicate copies.

#### Proposed Architecture



#### 1. SAN Integration

SAN contains enterprise block-storage data.

A connector or gateway can expose the required files/data to the ingestion layer.

Only required datasets should be transferred rather than copying the entire SAN.

#### 2. NAS Integration

NAS files can be collected using file-system-based ingestion.

NiFi can monitor directories and transfer only new or changed files.

#### 3. Operational Databases

Databases such as MySQL, PostgreSQL, or Oracle can be connected using database connectors.

**CDC (Change Data Capture)** is preferred because only changed records are transferred.

#### **4. Apache NiFi**

NiFi acts as the central ingestion and routing layer.

It provides:

- Data routing
- Transformation
- Validation
- Monitoring
- Back-pressure
- Provenance tracking

#### **5. Deduplication**

To minimize duplication:

- Generate unique file/record IDs.
- Use checksums or hashes.
- Track source system and timestamp.
- Use CDC for databases.
- Avoid repeatedly copying unchanged files.

Example:

File → SHA-256 Hash → Compare → Store only if new/changed

#### **6. HDFS/Data Lake**

After validation, data can be stored in HDFS.

Data can be separated into:

/raw

/processed

/curated

/archive

Partitioning by date/source improves query performance.

## **7. Backup**

Important data should have reliable backup.

Possible strategy:

Primary HDFS → Backup/Secondary Storage

Critical metadata should also be backed up separately.

## **8. Reliability**

Reliability is provided through:

- HDFS replication
- NiFi provenance
- Retry mechanisms
- Checkpointing
- Database transaction tracking
- Backup storage

## **9. Security**

Data movement should use:

- TLS encryption
- Authentication
- Access control
- Kerberos where appropriate

## **Justification**

NiFi provides controlled ingestion and monitoring, CDC minimizes unnecessary database transfers, checksums reduce duplicate files, HDFS provides distributed storage, and backup storage protects against data loss.

## 5. Question

**Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.**

### Answer

An end-to-end Hadoop ecosystem should integrate data ingestion, secure storage, processing, resource management, and analytics while providing high availability.

### Proposed Architecture

Data Sources



Apache NiFi / ETL



Secure Data Transfer



HDFS



YARN



MapReduce / Spark / Hive



Analytics / BI

### 1. Data Ingestion

Data can come from:

- Databases
- Applications
- IoT devices

- Files
- Logs

**Apache NiFi** can collect, route, transform, and monitor the data.

## **2. Secure Data Movement**

Data should be protected during transfer using:

- TLS/SSL
- Authentication
- Kerberos
- Access control
- Encrypted communication channels

NiFi can securely communicate with downstream Hadoop services.

## **3. HDFS Storage**

HDFS stores the incoming data across multiple DataNodes.

For critical data:

**Replication Factor = 3**

This protects against DataNode failures.

## **4. High Availability**

NameNode high availability can be implemented using:

Active NameNode



Standby NameNode



JournalNodes

If the Active NameNode fails, the Standby NameNode can take over.

## **5. YARN Resource Management**

YARN manages cluster resources for:

- Spark
- MapReduce
- Hive
- Machine learning
- ETL jobs

Different queues can be created for different workloads.

## **6. Data Processing**

**MapReduce** can perform large batch operations.

**Apache Spark** can be used for faster iterative processing and machine learning.

**Hive** can provide SQL-based analysis over Hadoop data.

## **7. ETL Integration**

NiFi can perform initial:

- Filtering
- Routing
- Validation
- Transformation

Additional ETL can be performed using Spark or Hive.

## **8. Monitoring and Recovery**

The platform should monitor:

- DataNode health
- NameNode status
- YARN containers
- NiFi processors
- Data transfer failures

Failed transfers can be retried automatically.

## 9. Analytics Layer

Processed data can be exposed to:

- Hive
- Spark SQL
- BI tools
- Reporting applications

Example:

HDFS → Hive/Spark → Data Mart → Power BI/Tableau

## 10. Constraint Satisfaction

| Constraint | Design Solution |
|------------|-----------------|
|------------|-----------------|

|                      |                                                  |
|----------------------|--------------------------------------------------|
| Secure data movement | TLS, Kerberos, authentication and access control |
|----------------------|--------------------------------------------------|

|                   |                                           |
|-------------------|-------------------------------------------|
| High availability | NameNode HA, HDFS replication, monitoring |
|-------------------|-------------------------------------------|

|                      |                                 |
|----------------------|---------------------------------|
| NiFi/ETL integration | NiFi ingestion + Spark/Hive ETL |
|----------------------|---------------------------------|

|                |                                          |
|----------------|------------------------------------------|
| Fault recovery | Retries, replication, container recovery |
|----------------|------------------------------------------|

|                     |                        |
|---------------------|------------------------|
| Scalable processing | YARN + Spark/MapReduce |
|---------------------|------------------------|

|           |                           |
|-----------|---------------------------|
| Analytics | Hive/Spark SQL + BI tools |
|-----------|---------------------------|